

PMS Maran – Módulos Core Operativos para Replit (Django)

Este documento complementa el proyecto de PMS Maran en Replit. Define y aporta código base para los módulos operativos clave: - Reservas PRO (códigos internos, segmentos, empresas, duplicar reserva) - Planning PRO (colores por grupo, acceso a reservas) - Check-in / Check-out con resumen de cuenta - Housekeeping con vistas diferenciadas - Cargos / Cuenta (categorías detalladas y cargos por noche) - Dashboard diario ampliado - Roles y permisos (versión 1)

1) Reservas PRO: códigos internos, segmentos, empresas, duplicar

Se asume que ya existe Guest, Reservation y algún manejo de datos personales en el sistema actual. Aquí se agregan campos y lógica complementaria.

1.1 Nuevos campos en Guest (core/models.py)

```
class Guest(models.Model):
    SEGMENT_CHOICES = [
        ('LEISURE', 'Ocio'),
        ('CORP', 'Corporativo'),
        ('SPORT', 'Deportivo'),
        ('CONGRESS', 'Congresos / Eventos'),
        ('OTHER', 'Otros'),
    ]

    hotel = models.ForeignKey(Hotel, on_delete=models.CASCADE)
    internal_code = models.CharField(max_length=30, unique=True) # Código interno de huésped
    first_name = models.CharField(max_length=100)
    last_name = models.CharField(max_length=100)
    email = models.EmailField(blank=True, null=True)
    phone = models.CharField(max_length=50, blank=True, null=True)
    document_number = models.CharField(max_length=50, blank=True, null=True)
    country = models.CharField(max_length=50, blank=True, null=True)
    segment = models.CharField(max_length=10, choices=SEGMENT_CHOICES, default='LEISURE')

    def __str__(self):
        return f"{self.internal_code} - {self.last_name}, {self.first_name}"
```

1.2 Modelo Empresa / Cuenta Corporativa

```
class Company(models.Model):
    hotel = models.ForeignKey(Hotel, on_delete=models.CASCADE)
    internal_code = models.CharField(max_length=30, unique=True) # Código interno de empresa
    name = models.CharField(max_length=200)
    tax_id = models.CharField(max_length=50, blank=True, null=True)
    email = models.EmailField(blank=True, null=True)
    phone = models.CharField(max_length=50, blank=True, null=True)
    notes = models.TextField(blank=True, null=True)

    def __str__(self):
        return f"{self.internal_code} - {self.name}"
```

1.3 Enlazar Reservation con Company y Segmento

```

class Reservation(models.Model):
    # ... campos ya existentes ...
    company = models.ForeignKey(Company, on_delete=models.SET_NULL, null=True, blank=True)
    # segment se puede heredar del huésped, pero se deja campo para override si se desea
    segment = models.CharField(max_length=10, choices=Guest.SEGMENT_CHOICES, default='LEISURE')
    # resto igual...

```

1.4 Botón duplicar reserva (vista orientativa)

```

@login_required
def reservation_duplicate(request, reservation_id):
    hotel = request.user.userprofile.hotel
    original = get_object_or_404(Reservation, id=reservation_id, hotel=hotel)

    if request.method == 'POST':
        new_checkin = request.POST.get('checkin_date')
        new_checkout = request.POST.get('checkout_date')
        room_id = request.POST.get('room_id') or original.room_id

        room = get_object_or_404(Room, id=room_id, hotel=hotel)

        nights = (datetime.strptime(new_checkout, '%Y-%m-%d').date() -
                  datetime.strptime(new_checkin, '%Y-%m-%d').date()).days

        new_res = Reservation.objects.create(
            hotel=hotel,
            reservation_code=f"{original.reservation_code}-DUP",
            guest=original.guest,
            room_type=room.room_type,
            room=room,
            checkin_date=new_checkin,
            checkout_date=new_checkout,
            nights=nights,
            rate_plan=original.rate_plan,
            base_rate_per_night=original.base_rate_per_night,
            discount_type=original.discount_type,
            discount_value=original.discount_value,
            final_rate_per_night=original.final_rate_per_night,
            total_room_amount=original.final_rate_per_night * nights,
            status='CONFIRMED',
            source=original.source,
            notes=f"Duplicada de {original.reservation_code}",
            last_modified_by=request.user
        )

        return redirect('reservation_detail', new_res.id)

    rooms = Room.objects.filter(hotel=hotel)
    return render(request, 'core/reservation_duplicate.html', {
        'original': original,
        'rooms': rooms,
    })

```

2) Planning PRO: colores por grupos y acceso a reservas

Además de los estados por reserva, el planning debe diferenciar grupos con colores y permitir abrir la reserva desde la celda.

2.1 Color por grupo (campo en Group)

```
class Group(models.Model):
    # ... campos existentes ...
    color = models.CharField(max_length=7, default='#FFD700') # Código HEX, ej. #FF0000
```

2.2 Lógica de contexto en la vista de planning

En planning_view, al construir occupancy, incluir referencia al grupo si existe

```
links = GroupReservationLink.objects.select_related('group', 'reservation')
```

```
res_to_group = {grl.reservation_id: grl.group for grl in links}
```

```
for res in reservations:
    current = res.checkin_date
    while current < res.checkout_date:
        if start_date <= current <= days[-1]:
            key = current.strftime('%Y-%m-%d')
            cell_info = {
                'reservation': res,
                'status': res.status,
                'room_status': res.room.status,
                'group': res_to_group.get(res.id)
            }
            occupancy[res.room.id][key] = cell_info
            current += timedelta(days=1)
```

2.3 Nota de plantilla

En la plantilla planning.html se accederá a occupancy[room.id][date] mediante filtros de plantilla o tags personalizados, y se usará el color del grupo si existe, por ejemplo: style="background-color: {{ cell.group.color }}". Desde la celda se debe mostrar link a detalle de reserva si cell.reservation existe; si no, link para crear reserva.

3) Check-in / Check-out con resumen de cuenta

Se asume que cada reserva tiene su folio de cargos. El check-out debe mostrar un resumen de cuenta y saldo antes de finalizar.

3.1 Función auxiliar para obtener saldo de reserva

```
def get_reservation_balance(reservation):
    total_charges = sum(c.amount for c in reservation.charges.all())
    return total_charges
```

3.2 Vista de Check-out

```
@login_required
def reservation_checkout(request, reservation_id):
    hotel = request.user.userprofile.hotel
    reservation = get_object_or_404(Reservation, id=reservation_id, hotel=hotel)

    charges = reservation.charges.all().order_by('date', 'id')
    balance = get_reservation_balance(reservation)

    if request.method == 'POST':
        if balance != 0:
            messages.error(request, "La reserva tiene saldo pendiente. No se puede hacer check-out.")
            return redirect('reservation_checkout', reservation.id)

        reservation.status = 'CHECKED_OUT'
        reservation.last_modified_by = request.user
        reservation.save()

        room = reservation.room
        room.status = 'DIRTY'
        room.save()

        # Integración futura: facturación
        return redirect('dashboard')

    return render(request, 'core/reservation_checkout.html', {
        'reservation': reservation,
        'charges': charges,
        'balance': balance,
    })
```

4) Housekeeping: accesos y vista de gobernanta

Housekeeping tendrá un acceso distinto. La gobernanta debe ver resumen de habitaciones que ingresan, ocupadas, ya limpias, las que salen, y poder agregar comentarios.

4.1 Notas de housekeeping por habitación y día

```
class HousekeepingNote(models.Model):
    hotel = models.ForeignKey(Hotel, on_delete=models.CASCADE)
    room = models.ForeignKey(Room, on_delete=models.CASCADE)
    date = models.DateField()
    note = models.TextField()
    created_at = models.DateTimeField(auto_now_add=True)
    created_by = models.ForeignKey(User, on_delete=models.SET_NULL, null=True)
```

4.2 Vista principal para gobernanta

```
@login_required
def housekeeping_dashboard(request):
    hotel = request.user.userprofile.hotel
    today = date.today()

    rooms = Room.objects.filter(hotel=hotel).order_by('floor', 'number')
    arrivals = Reservation.objects.filter(hotel=hotel, checkin_date=today, status__in=['TENTATIVE', 'CONFIRMED'])
    departures = Reservation.objects.filter(hotel=hotel, checkout_date=today, status='CHECKED_IN')
    occupied = Reservation.objects.filter(
        hotel=hotel,
        status__in=['CONFIRMED', 'CHECKED_IN'],
        checkin_date__lte=today,
        checkout_date__gt=today
    )

    notes = HousekeepingNote.objects.filter(hotel=hotel, date=today)

    context = {
        'rooms': rooms,
        'arrivals': arrivals,
        'departures': departures,
        'occupied': occupied,
        'notes': notes,
        'today': today,
    }
    return render(request, 'core/housekeeping_dashboard.html', context)
```

5) Cargos / Cuenta: categorías y cargos por noche

Se ajustan categorías y se define que el cargo de la primera noche se genere al check-in y las siguientes noches por proceso diario.

5.1 Ajuste de categorías en Charge

```
class Charge(models.Model):
    CATEGORY_CHOICES = [
        ('ROOM', 'Room'),
        ('ROOM_SERVICE', 'Room Service'),
        ('RESTAURANT', 'Restaurant'),
        ('SPA', 'Spa'),
        ('OTHER', 'Other'),
    ]
    # resto de campos idénticos, solo se ajusta CATEGORY_CHOICES
```

5.2 Cargo de primera noche al check-in

```
@login_required
def reservation_checkin(request, reservation_id):
    hotel = request.user.userprofile.hotel
    reservation = get_object_or_404(Reservation, id=reservation_id, hotel=hotel)

    if request.method == 'POST':
        reservation.status = 'CHECKED_IN'
        reservation.last_modified_by = request.user
        reservation.save()

        room = reservation.room
        room.status = 'OCCUPIED'
        room.save()

        Charge.objects.create(
            hotel=hotel,
            reservation=reservation,
            description="Noche de alojamiento (1)",
            amount=reservation.final_rate_per_night,
            date=date.today(),
            category='ROOM',
            created_by=request.user
        )

        return redirect('dashboard')

    return render(request, 'core/reservation_checkin.html', {'reservation': reservation})
```

5.3 Proceso diario para cargos de noches siguientes (esquema)

```
def post_nightly_room_charges(hotel, target_date):
    reservations = Reservation.objects.filter(
        hotel=hotel,
        status='CHECKED_IN',
        checkin_date__lt=target_date,
        checkout_date__gt=target_date
    )
    for res in reservations:
        exists = Charge.objects.filter(reservation=res, date=target_date, category='ROOM').exists()
        if not exists:
            Charge.objects.create(
                hotel=hotel,
                reservation=res,
                description=f"Noche de alojamiento ({target_date})",
```

```
        amount=res.final_rate_per_night,  
        date=target_date,  
        category='ROOM',  
        created_by=None  
    )
```

6) Dashboard diario ampliado

El dashboard ya muestra ocupación, llegadas y otras métricas. Se agrega explícitamente el conteo de salidas del día.

```
def dashboard_view(request):
    hotel = request.user.userprofile.hotel
    today = date.today()

    rooms = Room.objects.filter(hotel=hotel)
    total_rooms = rooms.count()

    todays_reservations = Reservation.objects.filter(
        hotel=hotel,
        status__in=['CONFIRMED', 'CHECKED_IN'],
        checkin_date__lte=today,
        checkout_date__gt=today
    )
    occupied_rooms = todays_reservations.values_list('room_id', flat=True).distinct().count()

    arrivals = Reservation.objects.filter(hotel=hotel, checkin_date=today, status__in=['TENTATIVE', 'CONFIRMED'])
    departures = Reservation.objects.filter(hotel=hotel, checkout_date=today, status__in=['CHECKED_IN', 'CONFIRMED'])

    context = {
        'total_rooms': total_rooms,
        'occupied_rooms': occupied_rooms,
        'today': today,
        'arrivals': arrivals,
        'departures': departures,
    }
    return render(request, 'core/dashboard.html', context)
```


7) Roles y permisos – versión 1

Se recomienda usar Django Groups para roles: - Recepcion: crear/editar reservas, check-in/out, ver planning, aplicar descuentos limitados. - Housekeeping: ver planning, ver panel housekeeping, cambiar estados, agregar notas. - Gerencia: ver todo, modificar tarifas, cancelar reservas, ver reportes. - Dirección y Gerente: acceso full (superuser o grupo con todos los permisos). - Restaurant / Spa / Seguridad: generar cargos y ver huéspedes alojados básicos. El detalle fino de permisos se configura en Django Admin asignando permisos por modelo y vista.